SOFTWARE FAULT PREDICTION USING VARIOUS SUPERVISED MACHINE LEARNING TECHNIQUES

A thesis submitted in partial fulfillment of the requirements

for the award of the degree of

Master of Science in

Mathematics with Data Science

By

SANGEETA SAHOO

Roll Number-2345013

Regd. No-238426104729



SCHOOL OF APPLIED SCIENCES

KALINGA INSTITUTE OF INDUSTRIAL TECHNOLOGY

DEEMED TO BE UNIVERSITY

BHUBANESWAR

April 2025

SOFTWARE FAULT PREDICTION USING VARIOUS SUPERVISED MACHINE LEARNING TECHNIQUES

A thesis submitted in partial fulfillment of the requirements

for the award of the degree of

Master of Science in

Mathematics with Data Science

By

SANGEETA SAHOO

Roll Number-2345013

Regd. No-238426104729

Under the Supervision of

Dr. Vishal Pradhan



SCHOOL OF APPLIED SCIENCES

KALINGA INSTITUTE OF INDUSTRIAL TECHNOLOGY

DEEMED TO BE UNIVERSITY

BHUBANESWAR

April 2025

DECLARATION

I hereby declare that the work reported in M.Sc. thesis entitled **SOFTWARE FAULT PREDICTION USING VARIOUS SUPERVISED MACHINE LEARNING TECHNIQUES** is submitted in the **School of Applied Sciences, KIIT Deemed to be University, Bhubaneswar**, is a Bonafide record of my work carried out under the supervision of **Dr. Vishal Pradhan** have not submitted this work elsewhere for any other degree or diploma.

Student Signature

SANGEETA SAHOO

Roll Number-2345013

Regd. No-238426104729

CERTIFICATE

This is to certify that the thesis titled **SOFTWARE FAULT PREDICTION USING VARIOUS SUPERVISED MACHINE LEARNING TECHNIQUES** is the work done by **Sangeeta Sahoo (2345013 & 238426104729)** in partial fulfillment of the requirements for the award of the degree of **Master of Science** in **Mathematics with Data Science of the School of Applied Sciences, KIIT Deemed to be University, Bhubaneswar** under my/our supervision and this work has not been submitted elsewhere for a degree.

Dr. Vishal Pradhan

Supervisor

Prof. Prasanta Rath

Dean, School of Applied Sciences

Thesis Viva-voce held on _____

Internal Examiner

External Examiner

ABSTRACT

In an age where software drives innovation across industries, ensuring its reliability has become more critical than ever. Predicting software faults before deployment not only saves time and cost but also enhances user trust and system performance. This study applies supervised machine learning techniques to detect potential software defects at an early stage. Machine learning models were applied to software datasets made publicly available by NASA[20]. These datasets include various numerical values, known as software metrics, which describe different features of the code.

The first step in the process was data preprocessing. This involved carefully examining the data for any missing values. In one dataset, many values were missing in a specific column, and this was handled using a simple technique called mean imputation, where the missing entries are replaced with the average value of that column. This helps ensure that no useful data is lost while keeping the dataset consistent. In addition, the data was cleaned and simplified by selecting only the most useful features. This step, known as feature selection, helps the machine learning models run more efficiently and prevents them from learning from unnecessary or repeated information.

Four different types of classification models were used to build and test predictions: Logistic Regression, Support Vector Machine (SVM), Decision Tree, and Random Forest. These models were chosen because they are well-known and commonly used in machine learning. Each model was trained on the prepared data to learn how to recognize patterns that might point to software defects. The models were then tested to see how well they could predict faults in new data.

To measure performance, several evaluation tools were used. These include Precision, Recall, F1-Score, Mean Squared Error (MSE), and visual methods like the ROC Curve and AUC (Area Under the Curve). These metrics help compare the models and choose the one that works best.

Overall, this study shows that using proper data cleaning, smart feature selection, and well-known machine learning models can help detect software defects early. This can be very useful for software developers and companies to improve the quality of their software and reduce errors before release.

Keywords:

Logistic Regression, Support Vector Machine (SVM), Decision Tree Classifier, Random Forest Classifier, Software Fault Prediction

ACKNOWLEDGEMENT

I would like to take this opportunity to express my profound gratitude to all those who have extended their support and encouragement throughout the course of this work and contributed directly or indirectly to the successful completion of this report.

First and foremost, I am deeply indebted to my supervisor, Dr. Vishal Pradhan, for their invaluable guidance, constructive feedback, and continuous encouragement. Their expertise, insightful suggestions, and patient supervision have been a constant source of motivation and have played a pivotal role in shaping the quality and depth of this report. I am sincerely thankful for the time and effort they devoted to helping me understand complex concepts and refine my research.

I would also like to thank the faculty members and staff of the Department of Mathematics, KIIT University, for providing a strong academic foundation and a supportive environment that encouraged exploration and critical thinking. The resources and facilities offered by the department were instrumental in carrying out this study effectively.

My sincere thanks go to the authors and researchers whose published works formed the basis of my literature review. Their comprehensive studies, insightful findings, and methodological approaches provided significant direction and enriched the understanding of the problem discussed in this report.

Lastly, I wish to express heartfelt appreciation to my peers and friends, who provided encouragement, helpful discussions, and support during the various stages of this work. Their companionship made this academic journey more enjoyable and less stressful.

TABLE OF CONTENTS

ABSTRACT.....i

ACKNOWLEDGEMENT.....iii

TABLE OF CONTENTS.....iv

CHAPTER 1 INTRODUCTION……..…**1**

CHAPTER 2 LITERATURE REVIEW……**3**

CHAPTER 3 PROBLEM FORMULATION & PROPOSED WORK……....**5**

CHAPTER 4 DATA ANALYSIS

4.1. Dataset Details……..8

4.2. Feature Selection……....9

4.3. Classification……...11

CHAPTER 5 COMPUTATIONAL ANALYSIS

5.1. Correlation Matrix and Heatmap……...…….13

5.2. Performance Metrics……...……....24

5.3. Comparison table.……....28

5.4. ROC Curve and AUC……...……...30

CHAPTER 6 CONCLUSION…….**32**

CHAPTER 1
INTRODUCTION

Understanding Software Faults and Their Impact

In the field of software technology, a software fault occurs when a software system or component deviates from its expected behaviour, leading to incorrect outputs, system crashes, or unintended operations[1]. These faults can stem from various sources, including coding errors, design flaws, or unforeseen interactions within complex systems[2]. The consequences of software faults can range from minor inconveniences to catastrophic events, especially in safety-critical domains like aerospace, healthcare, and finance[3].

As software systems become increasingly integral to daily operations across industries, the prediction of software faults has emerged as a critical area of focus[4]. By anticipating potential faults before they manifest, organizations can proactively address vulnerabilities, enhance system reliability, and moderate risks associated with software malfunctions[5]. This predictive approach not only safeguards end-users but also preserves organizational reputation and reduces financial liabilities.

Software faults have caused significant disruptions in various sectors. In aviation systems, glitches in flight control or navigation software can jeopardize passenger safety and result in operational delays[6]. In the healthcare sector, software faults in medical devices may lead to incorrect treatments or missed alerts, putting patients' lives at risk[7]. The banking and finance industries are also vulnerable, as bugs in financial software can result in erroneous transactions, data breaches, or unexpected downtimes[8]. Similarly, faults in automotive software can affect vehicle control, potentially leading to dangerous driving conditions. In public services, software issues may delay essential processes, compromising service delivery. Even in consumer applications, bugs can impair usability and lead to user dissatisfaction or financial losses.

The Role of Prediction and Challenges Ahead

The motivation to predict software faults stems from the need to enhance system reliability, ensure user safety, optimize maintenance efforts, and inform better design practices[9]. By identifying potential fault points early in the development lifecycle, organizations can reduce system downtime and intervene before problems escalate[10]. This is particularly vital in domains such as healthcare and aviation, where a fault could have life-threatening consequences[11]. Predictive insights also help streamline maintenance by targeting only the necessary areas, conserving resources and lowering costs. Furthermore, understanding fault patterns can influence the design and architecture of future software systems, making them more resilient to similar issues.

Effective prediction of software faults relies heavily on the availability and quality of relevant data[12]. Operational data, including logs and performance metrics, help capture system behaviour under different conditions. Historical fault data, such as crash reports and error messages, provide a record of past issues and highlight recurring patterns. Testing data from unit, integration, and system-level tests reveal weaknesses in the software and identify areas prone to faults. Machine learning and statistical techniques applied to these data sources can significantly enhance the accuracy of fault prediction models[13].

However, several challenges persist. Poor data quality or incomplete datasets can weaken model performance and reduce prediction accuracy. The continuous evolution of software systems also means that models must be updated frequently to remain effective[14]. Additionally, the complex interdependencies within modern software make it difficult to isolate specific fault causes[15]. Human factors, such as differences in user behaviour or operational practices, further complicate the task by introducing unpredictable elements into the system. Despite these challenges, the field continues to progress, aiming to create robust, interpretable, and reliable predictive models that contribute to safer and more efficient software systems.

CHAPTER 2
LITERATURE REVIEW

Over the years, a significant amount of research has been conducted in the field of software defect prediction, with various approaches proposed to improve the accuracy and efficiency of identifying fault-prone modules. Each method contributes valuable insight, but most also come with practical limitations that leave room for further improvement.

Khoshgoftaar et al. [1] pioneered the use of statistical models such as logistic regression and decision trees on software metrics to predict defects in telecommunication systems. While their approach introduced the potential of predictive analytics in software engineering, it was limited by an over-reliance on linear relationships and small dataset sizes, which affected generalizability.

Nagappan and Ball [15] introduced a model using code churn metrics and change history from version control systems. They emphasized that defect prediction is not solely dependent on structural metrics but also on how frequently code changes. However, their model underperformed when applied to projects with limited version control history or static modules, and thus wasn't suitable for all types of systems.

Lessmann et al. [3] conducted a large-scale comparison of machine learning classifiers (e.g., SVM, Naive Bayes, Decision Tree, Random Forest) on defect prediction tasks. Although Random Forest and SVM performed well, their study lacked rigorous feature selection and data cleaning steps, which may have inflated performance metrics due to noisy inputs or multicollinearity.

More recently, Malhotra [4] reviewed over 60 defect prediction models and concluded that most studies fail to address class imbalance, redundant features, or interpretability of results. Many models operated as black boxes and did not provide actionable insights for developers, which is a critical requirement in industrial applications.

Despite the depth of research, several gaps remain in the existing literature. Many studies utilize datasets containing missing or noisy data without clearly explaining the methods

used to handle these issues. Additionally, few researchers apply outlier detection techniques prior to model training, even though outliers can significantly skew the learning process. The correlation between features is often overlooked, leading to models that are trained on highly redundant data. Furthermore, the majority of models are evaluated using only a single dataset, raising concerns about their robustness and generalizability across different software projects.

In contrast, the present work addresses these limitations through a series of systematic approaches. It applies Interquartile Range (IQR)-based outlier detection along with mean imputation to effectively handle missing values in the datasets. To reduce dimensionality and address multicollinearity, correlation matrices and heatmaps are utilized. Moreover, the study enhances robustness by comparing multiple machine learning models including Random Forest, Support Vector Machine (SVM), Logistic Regression, and Decision Tree, across four real-world datasets. Clear and informative visualizations such as ROC curves, confusion matrices, and F1-score bar plots are employed to improve the interpretability of the model results.

Thus, this study contributes to the field by presenting a comprehensive, well-structured, and reproducible machine learning pipeline that improves the quality and generalizability of software defect prediction.

CHAPTER 3

PROBLEM FORMULATION AND PROPOSED WORK

Problem Formulation

Software systems are prone to faults that can lead to faults, significantly affecting performance, reliability, and user satisfaction. Identifying defective modules in the early stages of development is crucial to improving software quality and reducing maintenance costs. This project focuses on predicting software defects using machine learning techniques.

The primary objective is to formulate a binary classification problem where each software module is labeled as either defective or non-defective based on a set of software metrics. The input features consist of various static code attributes (such as complexity measures, line counts, and density metrics), while the output is a binary label indicating the presence or absence of a defect.

To solve this problem, the following steps are undertaken:

- Data preprocessing: Handling missing values, detecting outliers, and selecting relevant features.
- Model training: Applying supervised learning algorithms like Logistic Regression, SVM, Decision Tree, and Random Forest.
- Model evaluation: Assessing performance using metrics such as Precision, Recall, F1-score, ROC-AUC[5], and Mean Squared Error (MSE).

This formulation allows for the development of predictive models capable of generalizing well on unseen data and aiding software engineers in identifying risky components before deployment.

Proposed Work

This project aims to predict software fault using supervised machine learning techniques, based on static code metrics extracted from real-world software datasets. Building on the existing implementation, which includes exploratory data analysis, correlation-based feature reduction, and baseline model training, the proposed work focuses on improving model performance, enhancing reliability, and ensuring better generalizability of the results.

The initial stage of the project involved identifying and removing features that were highly correlated (correlation ≥ 0.9), as they introduced redundancy and could negatively affect the learning process. This manual dimensionality reduction step ensured that the models trained on cleaner, more independent data, reducing the risk of overfitting and improving interpretability.

The proposed work will extend this by evaluating how different preprocessing strategies affect classification outcomes. Outlier detection using the Interquartile Range (IQR)[15] has already been integrated into the workflow, but further refinement will focus on balancing the trade-off between removing noise and retaining valuable edge cases.

Additionally, the project will continue to rely on four core classifiers like Logistic Regression, Decision Tree, Support Vector Machine (SVM), and Random Forest, which have already shown promising results across the datasets. These models will be further evaluated using updated performance metrics including Accuracy, Precision, Recall, F1-Score, and visual tools like Confusion Matrices and ROC Curves. Comparative analysis across the four datasets (DATASET 1, DATASET 4, DATASET 2, DATASET 3) will help determine the most robust model in terms of both consistency and predictive power. Another key focus will be on ensuring that all experiments and results are reproducible and systematic. The existing modelling steps will be organized into a streamlined, automated

process like combining preprocessing, feature selection, model training, and evaluation. This pipeline-based structure will allow for easier experimentation, consistent application of techniques across datasets, and a cleaner comparison of model outputs. Ultimately, the proposed enhancements are aimed at improving the accuracy and reliability of fault prediction models while maintaining transparency and ease of interpretation. By combining statistical understanding with practical implementation, the project seeks to provide valuable insights into software quality assurance and support informed decision-making during the development cycle.

Outlier values that significantly deviate from the normal range were identified using the Interquartile Range (IQR) method[14]. These data points were carefully analysed to decide whether they should be removed or retained. The goal is to eliminate noise that can skew the learning process without discarding information that might contribute to meaningful patterns. Ongoing efforts include visual inspection and impact analysis to ensure the treatment of these values supports overall model stability and performance.

In addition to this, missing data was addressed as part of the cleaning process. Few missing values were identified in one of the datasets and were handled using a simple imputation method based on the mean of the respective column. The data types present in the datasets and appropriate conversions were ensured during preprocessing. The overall workflow, including feature selection, data cleaning, and initial modelling, has been structured systematically to support further improvements and consistent evaluation.

CHAPTER 4
DATA ANALYSIS

4.1. Dataset Details

The study utilizes various datasets to analyse and predict software fault. Four distinct datasets are incorporated into this research, each providing valuable insights into software metrics and defect prediction. These datasets are gathered from open-source repositories, ensuring reproducibility and transparency in the research.

The datasets used in this work are:

I. **DATASET 1 Dataset**: This dataset comprises software metrics and corresponding defect labels. It contains 22 columns and 2,107 rows, providing a range of attributes that capture essential software characteristics.

II. **DATASET 2 Dataset**: The DATASET 2 dataset includes detailed software metrics along with defect labels. It features 38 columns and 403 rows, helping analyse more intricate patterns in software faults.

III. **DATASET 3 Dataset**: The DATASET 3 dataset, similar to the other datasets, contains software metrics and defect labels. It consists of 39 columns and 17,186 rows, offering a comprehensive dataset for fault prediction analysis.

IV. **DATASET 4 Dataset**: Like the other datasets, the DATASET 4 dataset provides software metrics with corresponding defect labels. It has 37 columns and 9,466 rows, contributing to the overall analysis of fault prediction.

Each of these datasets contains a unique set of input parameters, with the DATASET 1 dataset featuring 22 columns, the DATASET 2 dataset providing 38 columns, the DATASET 3 dataset containing 39 columns, and the DATASET 4 dataset having 37 columns. The data from these sources is divided into training and testing sets, where the older versions of the data are used for training the model, and the newer versions are reserved for testing. This approach ensures that the models are evaluated on their ability to generalize to new, unseen data.

4.2. Feature Selection

Feature selection is a crucial step in improving the performance of the software fault prediction model. It involves identifying and selecting the most relevant input features (metrics) while eliminating irrelevant or redundant ones. This process helps reduce the complexity of the model, improves computational efficiency, and enhances the model's generalization ability. Various techniques, such as correlation analysis, recursive feature elimination, and tree-based methods[9], are employed to select the most significant features that contribute to accurate fault predictions.

4.2.1. Data Types in the Datasets

Understanding data types is an important step before feature selection. Data types define the nature of data in each column and influence how it is processed. Common data types include:

- **int64**: Integer values (whole numbers)
- **float64**: Floating-point numbers (decimal values)
- **object**: Text or mixed data types (not applicable here)

In the datasets used for this project, the features are mainly of int64 and float64 types, representing various software metrics.

4.2.2. Handling Missing Values

During preprocessing, missing values were identified using Python code. Specifically, in the DATASET 2 dataset, the column DECISION_DENSITY was found to have 139 missing values.

Common techniques for handling missing values include:

- Dropping missing entries
- Imputing with mean, median, or mode
- Predictive imputation using models

For this project, the mean imputation method was used are the missing values in DECISION_DENSITY were replaced with the mean of the existing values in that column.

4.2.3. Outliers and Their Types

Outliers are values that differ significantly from the majority of data and can negatively affect model performance. Types of outliers include:

- **Point Outliers**: Individual values that are unusually high or low.
- **Contextual Outliers**: Values that are normal in one context but abnormal in another (e.g., time series).
- **Collective Outliers**: A group of values that are unusual when considered together.

Identifying and handling outliers ensures more stable and accurate predictions. Techniques like Z-score analysis, box plots, and IQR methods are often used.

By addressing missing values, understanding data types, and managing outliers, feature selection becomes more effective and leads to a more robust software fault prediction model.

4.3. Classification

Classification is a type of supervised machine learning where the goal is to assign input data into predefined categories or classes. In the context of software engineering, a common classification task is to predict whether a software component is defective (1) or not defective (0) based on its code metrics.

In this project, we explore and compare several well-known classification models to predict software defects using static code metrics. Each classifier brings a different approach to decision-making and offers unique advantages in handling the structure and complexity of the dataset.

4.3.1. Logistic Regression

A statistical method that uses a sigmoid function to estimate the probability that a given input belongs to a certain class. It is simple yet effective for binary classification tasks.

Real-life Example:
Predicting whether an email is spam (1) or not spam (0) based on words in the email.

Software Fault Example:
Predicting whether a software function is defective (1) or clean (0) based on complexity and line count metrics.

4.3.2. Support Vector Machine (SVM)

SVM finds the best boundary (hyperplane) that separates data points of different classes, maximizing the margin between them. It's effective for high-dimensional and complex datasets.

Real-life Example:
Classifying whether a handwritten digit is a '3' or '8' based on pixel values in an image.

Software Fault Example:

Classifying a function as defective or non-defective based on Halstead metrics and cyclomatic complexity.

4.3.3. Decision Tree Classifier

A model that makes decisions by splitting data into branches based on feature values. It mimics human decision-making and is easy to interpret.

Real-life Example:

A loan officer deciding to approve a loan based on age, income, and credit score (yes/no decision at each branch).

Software Fault Example:

A decision tree might classify a module as defective if:

- LOC_TOTAL > 1000 AND
- CYCLOMATIC_COMPLEXITY > 10

4.3.4. Random Forest Classifier

An ensemble of multiple decision trees where each tree votes for a class, and the majority vote becomes the prediction. It reduces overfitting and increases accuracy.

Real-life Example:

Multiple doctors independently diagnosing a patient, and the final decision is based on the majority.

Software Fault Example:

Multiple decision trees each analyse a software module's metrics and vote on whether it's defective, providing a reliable final decision.

CHAPTER 5

COMPUTATIONAL ANALYSIS

5.1. Correlation Matrix and Heatmap:

In machine learning and data analysis, understanding the relationship between features is essential for building efficient and accurate predictive models. One of the most widely used techniques for analysing feature relationships is the correlation matrix, which is often visualized using a heatmap.

5.1.1. Correlation Matrix:

A correlation matrix is a square matrix that shows the pairwise correlation coefficients between all numerical variables in a dataset. Each cell in the matrix contains a value that indicates the strength and direction of the linear relationship between two variables.

The most commonly used correlation coefficient is Pearson's correlation coefficient (r), which ranges from:

- **+1**: perfect positive correlation
- **0**: no linear correlation
- **–1**: perfect negative correlation

Mathematically, Pearson's correlation coefficient between variables X and Y is given by:

$$r = \frac{Cov\ (X, Y)}{\sigma_X . \sigma_Y}$$

Where:

- $Cov\ (X, Y)$ is the covariance between X and Y
- σ_X, σ_Y are the standard deviations of X and Y

5.1.2. Heatmap:

A heatmap is a graphical representation of data where individual values are displayed as colours. When applied to a correlation matrix, a heatmap uses colour intensity to represent the magnitude and direction of correlation between variables:

- Strong positive correlation (close to +1) is often shown in dark red
- Strong negative correlation (close to –1) is shown in deep blue
- Values near 0 (weak or no correlation) appear in light or neutral colours

This visual form helps users quickly identify:

- Highly correlated feature pairs (which may introduce redundancy)
- Uncorrelated or independent features
- Patterns or clusters of related variables

5.1.3. Relevance to This Project:

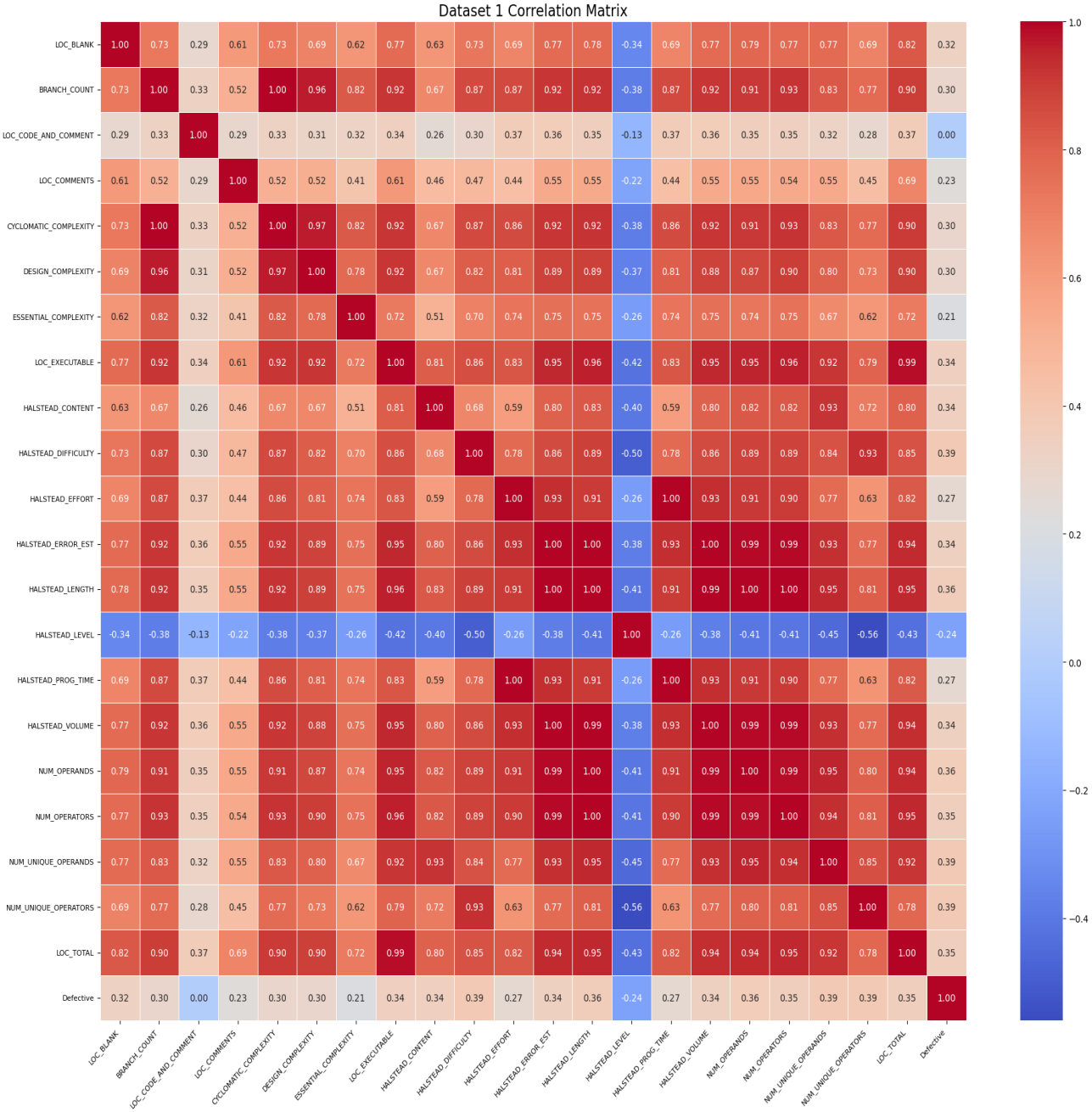
In the context of software fault prediction, a heatmap of the correlation matrix was used to:

- Detect highly correlated code metrics, such as strong relationships between HALSTEAD_LENGTH, NUM_OPERANDS, and HALSTEAD_VOLUME
- Identify and remove features with a correlation of 0.9 or above, to reduce multicollinearity and simplify the model
- Support feature selection and dimensionality reduction, ensuring that the predictive models are not biased or overwhelmed by redundant data

5.1.4. Key Terms Related to Correlation and Heatmaps

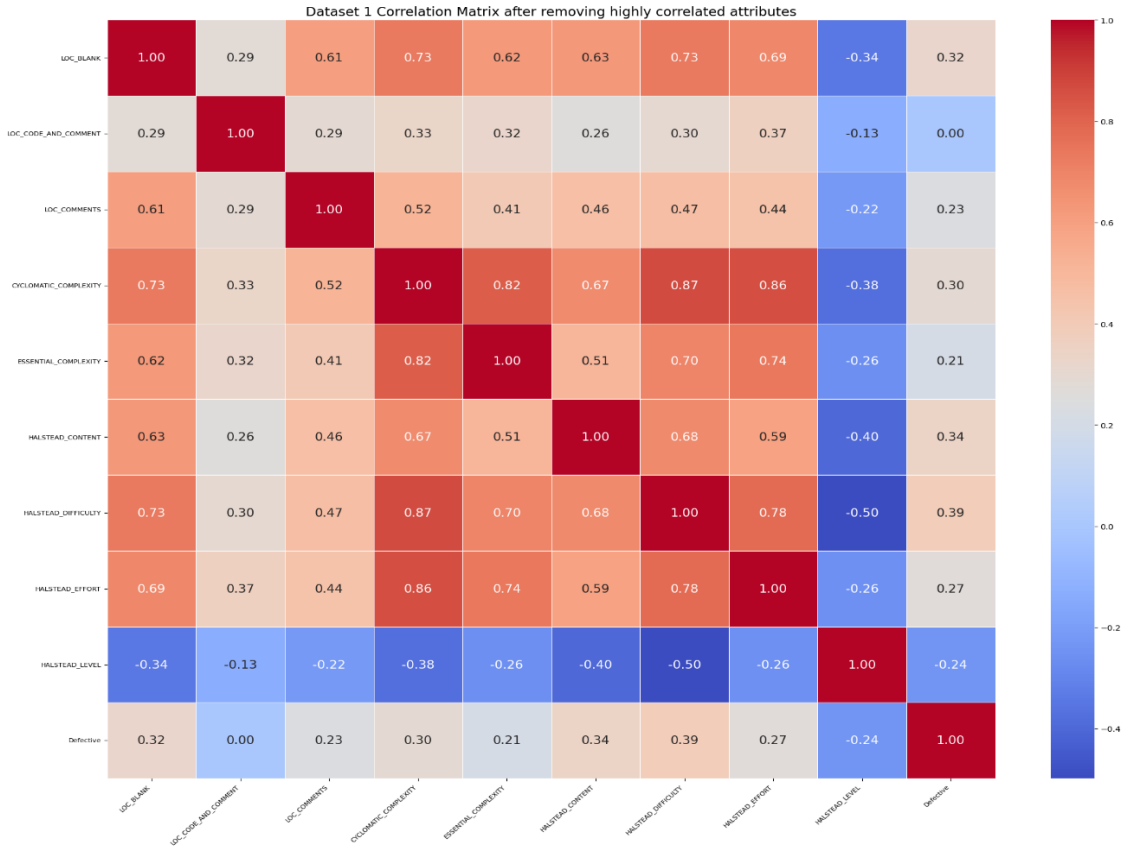
- **Multicollinearity**: A condition where two or more independent variables are highly correlated, which can distort model interpretation and reduce predictive power.
- **Dimensionality Reduction**: The process of reducing the number of input variables while preserving most of the information. Correlation analysis is one such approach.
- **Feature Engineering**: The process of selecting, transforming, or creating input features to improve model performance.

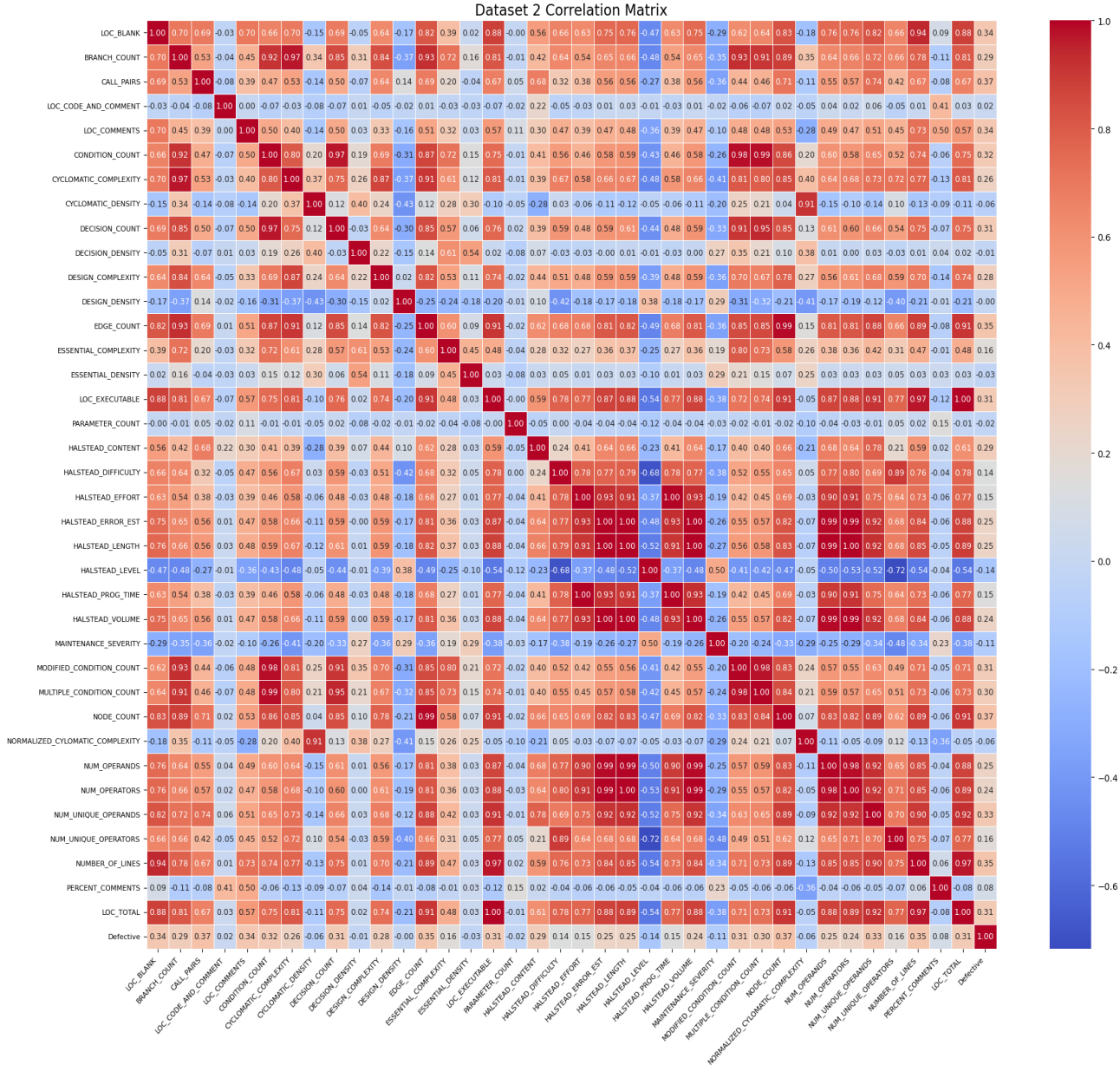
5.1.5. **Heatmap depicting correlation matrix before and after removing highly correlated attributes**



GROUP	KEEP	DROP
HALSTEAD_PROG_TIME, HALSTEAD_EFFORT	HALSTEAD_EFFORT	HALSTEAD_PROG_TIME
HALSTEAD_ERROR_EST, HALSTEAD_VOLUME, HALSTEAD_LENGTH, NUM_OPERANDS, NUM_OPERATORS	HALSTEAD_VOLUME	HALSTEAD_ERROR_EST, HALSTEAD_LENGTH, NUM_OPERANDS, NUM_OPERATORS
LOC_EXECUTABLE, LOC_TOTAL	LOC_EXECUTABLE	LOC_TOTAL
NUM_UNIQUE_OPERANDS, NUM_UNIQUE_OPERATORS	NUM_UNIQUE_OPERANDS	NUM_UNIQUE_OPERATORS
DESIGN_COMPLEXITY, CYCLOMATIC_COMPLEXITY ***(based on typical pairing)***	CYCLOMATIC_COMPLEXITY	DESIGN_COMPLEXITY
BRANCH_COUNT, CONDITION_COUNT ***(assumed common pairing)***	CONDITION_COUNT	BRANCH_COUNT

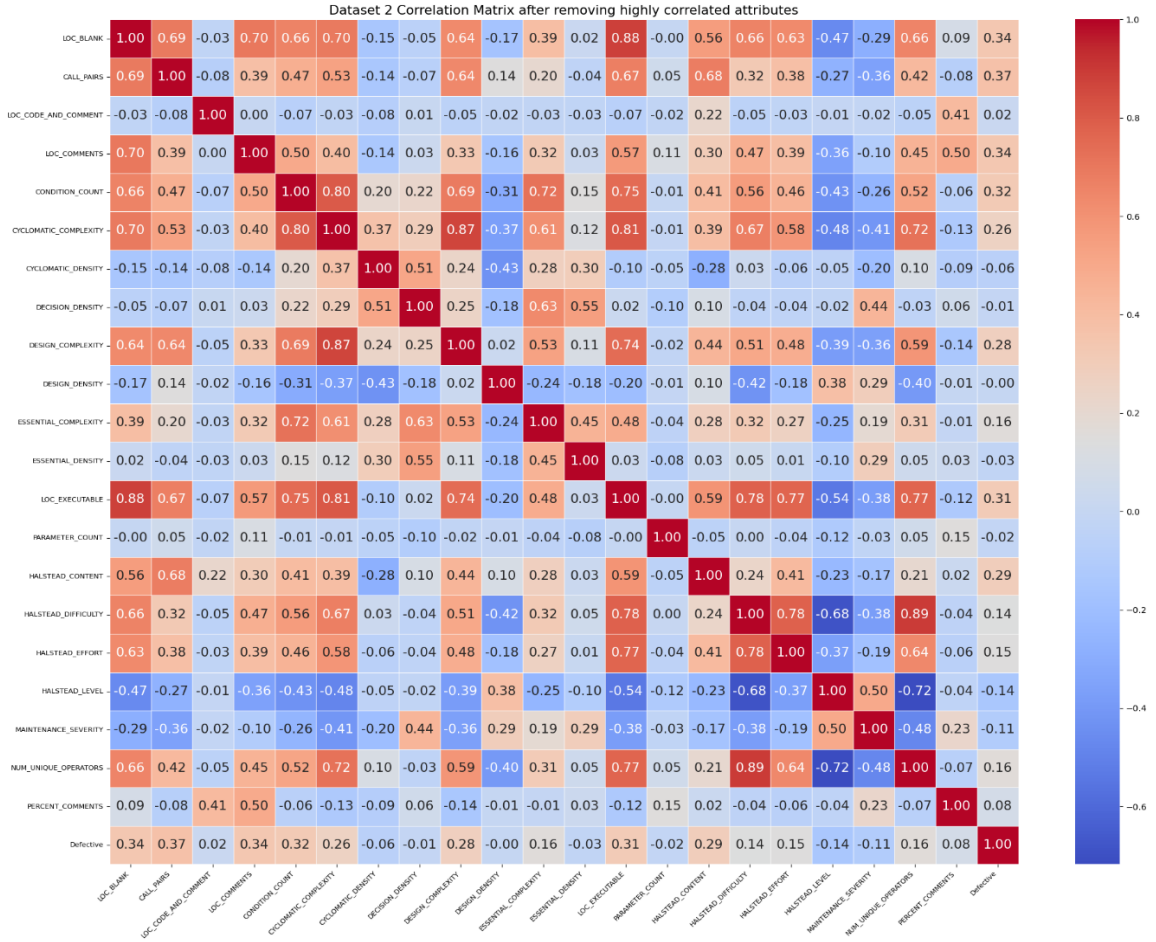
Table 1: *Feature Reduction Based on Correlation Analysis for Dataset 1*

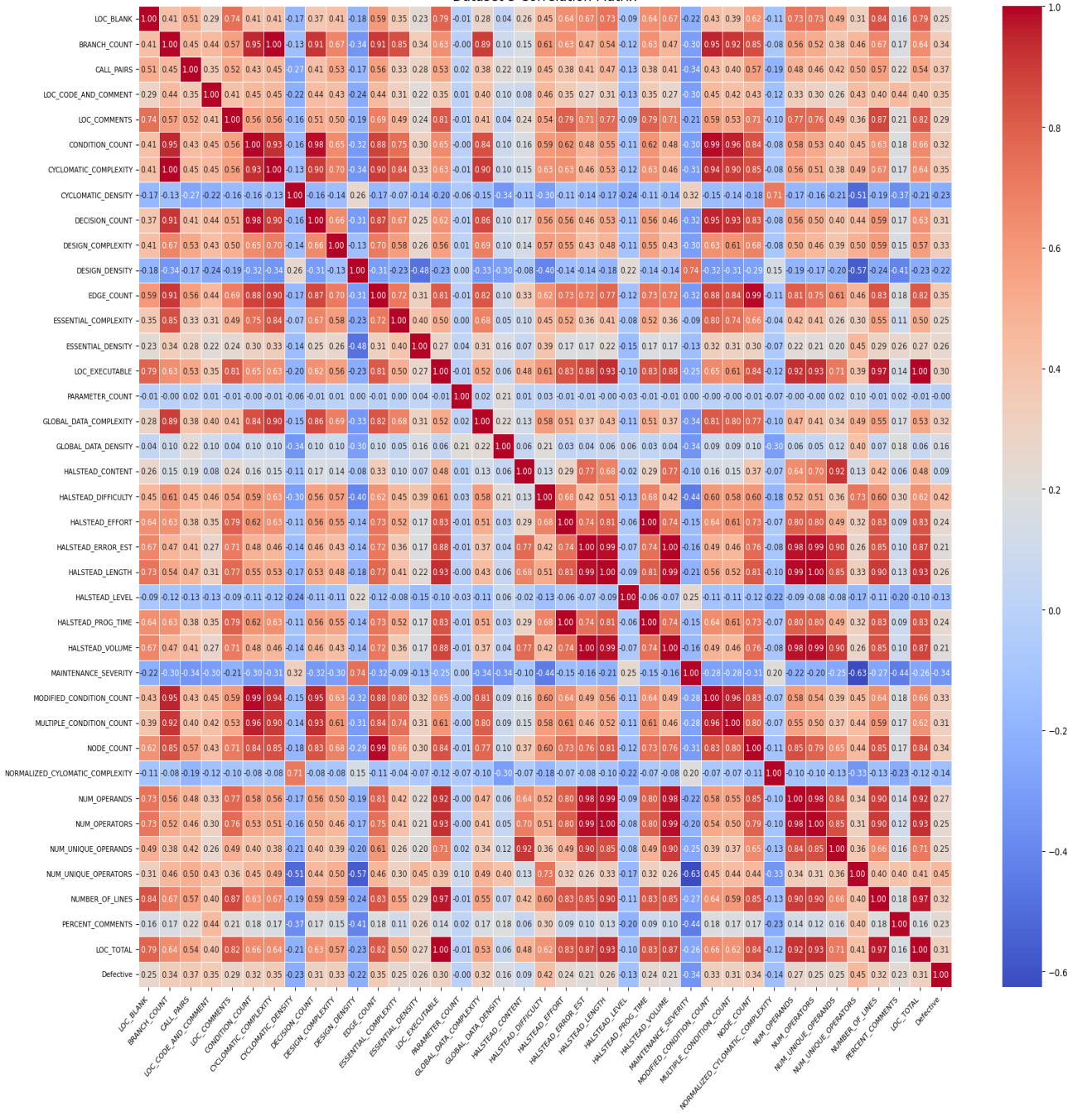



Dataset 2 Correlation Matrix
LOC_BLANK
BRANCH_COUNT
CALL_PAIRS
LOC_CODE_AND_COMMENT
LOC_COMMENTS
CONDITION_COUNT
CYCLOMATIC_COMPLEXITY
CYCLOMATIC_DENSITY
DECISION_COUNT
DECISION_DENSITY
DESIGN_COMPLEXITY
DESIGN_DENSITY
EDGE_COUNT
ESSENTIAL_COMPLEXITY
ESSENTIAL_DENSITY
LOC_EXECUTABLE
PARAMETER_COUNT
HALSTEAD_CONTENT
HALSTEAD_DIFFICULTY
HALSTEAD_EFFORT
HALSTEAD_ERROR_EST
HALSTEAD_LENGTH
HALSTEAD_LEVEL
HALSTEAD_PROG_TIME
HALSTEAD_VOLUME
MAINTENANCE_SEVERITY
MODIFIED_CONDITION_COUNT
MULTIPLE_CONDITION_COUNT
NODE_COUNT
NORMALIZED_CYLOMATIC_COMPLEXITY
NUM_OPERANDS
NUM_OPERATORS
NUM_UNIQUE_OPERANDS
NUM_UNIQUE_OPERATORS
NUMBER_OF_LINES
PERCENT_COMMENTS
LOC_TOTAL
Defective
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
−0.4
−0.6

GROUP	KEEP	DROP
HALSTEAD_EFFORT, HALSTEAD_PROG_TIME, HALSTEAD_LENGTH, HALSTEAD_VOLUME, HALSTEAD_ERROR_EST, NUM_OPERATORS, NUM_OPERANDS, NUM_UNIQUE_OPERANDS	HALSTEAD_EFFORT	HALSTEAD_PROG_TIME, HALSTEAD_LENGTH, HALSTEAD_VOLUME, HALSTEAD_ERROR_EST, NUM_OPERATORS, NUM_OPERANDS, NUM_UNIQUE_OPERANDS
LOC_EXECUTABLE, LOC_TOTAL, NUMBER_OF_LINES, EDGE_COUNT, NODE_COUNT	LOC_EXECUTABLE	LOC_TOTAL, NUMBER_OF_LINES, EDGE_COUNT, NODE_COUNT
CONDITION_COUNT, MODIFIED_CONDITION_COUNT, MULTIPLE_CONDITION_COUNT, DECISION_COUNT, BRANCH_COUNT	CONDITION_COUNT	MODIFIED_CONDITION_COUNT, MULTIPLE_CONDITION_COUNT, DECISION_COUNT, BRANCH_COUNT
CYCLOMATIC_DENSITY, NORMALIZED_CYLOMATIC_COMPLEXITY	CYCLOMATIC_DENSITY	NORMALIZED_CYLOMATIC_COMPLEXITY

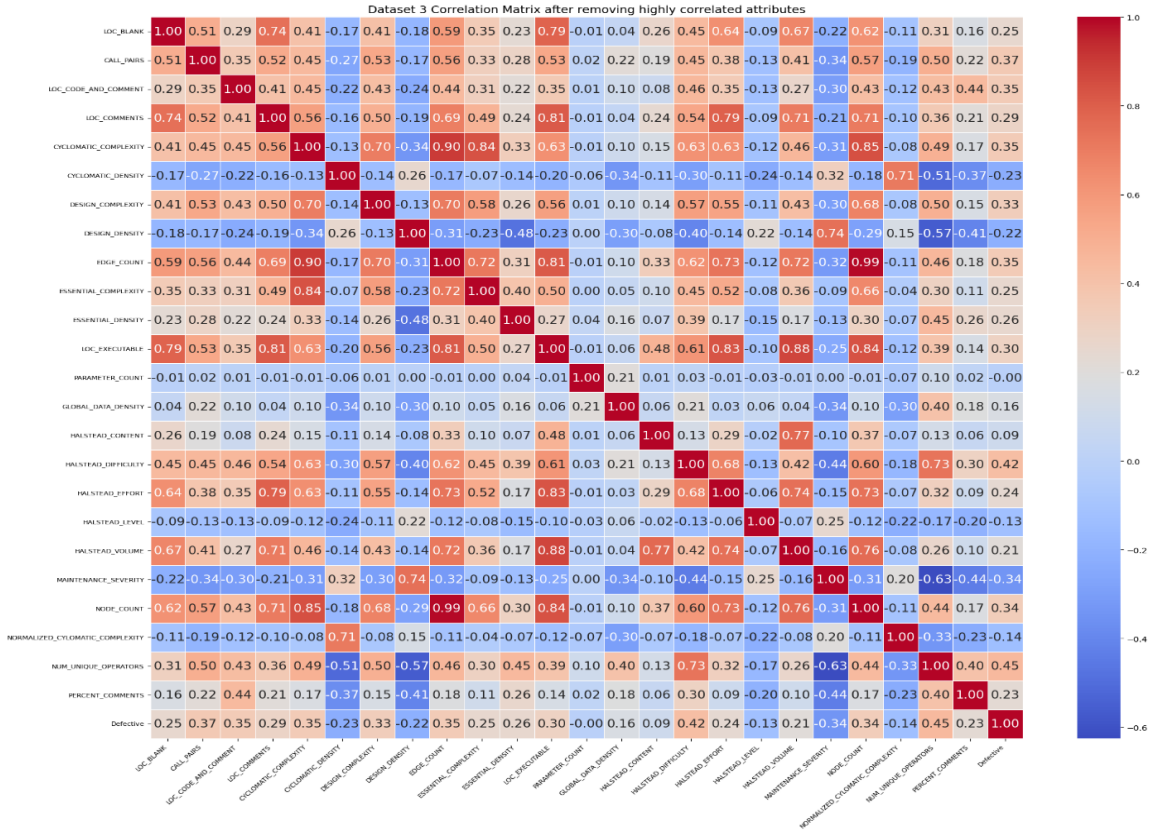
Table 2: *Feature Reduction Based on Correlation Analysis for Dataset 2*

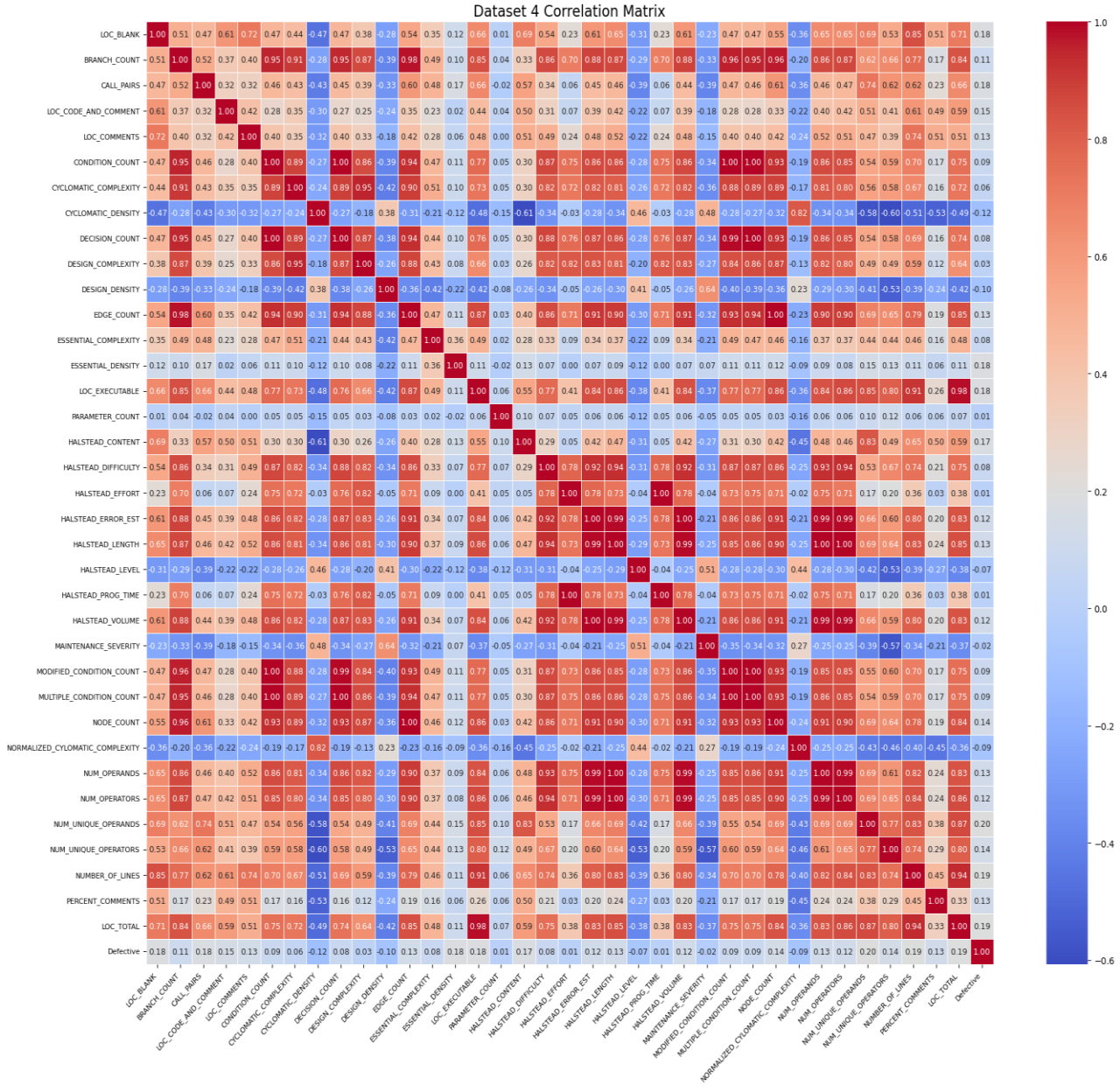



Dataset 3 Correlation Matrix
LOC_BLANK
BRANCH_COUNT
CALL_PAIRS
LOC_CODE_AND_COMMENT
LOC_COMMENTS
CONDITION_COUNT
CYCLOMATIC_COMPLEXITY
CYCLOMATIC_DENSITY
DECISION_COUNT
DESIGN_COMPLEXITY
DESIGN_DENSITY
EDGE_COUNT
ESSENTIAL_COMPLEXITY
ESSENTIAL_DENSITY
LOC_EXECUTABLE
PARAMETER_COUNT
GLOBAL_DATA_COMPLEXITY
GLOBAL_DATA_DENSITY
HALSTEAD_CONTENT
HALSTEAD_DIFFICULTY
HALSTEAD_EFFORT
HALSTEAD_ERROR_EST
HALSTEAD_LENGTH
HALSTEAD_LEVEL
HALSTEAD_PROG_TIME
HALSTEAD_VOLUME
MAINTENANCE_SEVERITY
MODIFIED_CONDITION_COUNT
MULTIPLE_CONDITION_COUNT
NODE_COUNT
NORMALIZED_CYLOMATIC_COMPLEXITY
NUM_OPERANDS
NUM_OPERATORS
NUM_UNIQUE_OPERANDS
NUM_UNIQUE_OPERATORS
NUMBER_OF_LINES
PERCENT_COMMENTS
LOC_TOTAL
Defective
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
−0.4
−0.6

GROUP	KEEP	DROP
HALSTEAD_EFFORT, HALSTEAD_PROG_TIME	HALSTEAD_EFFORT	HALSTEAD_PROG_TIME
HALSTEAD_VOLUME, HALSTEAD_ERROR_EST, HALSTEAD_LENGTH, NUM_OPERATORS, NUM_OPERANDS	HALSTEAD_VOLUME	HALSTEAD_ERROR_EST, HALSTEAD_LENGTH, NUM_OPERATORS, NUM_OPERANDS
LOC_EXECUTABLE, LOC_TOTAL, NUMBER_OF_LINES	LOC_EXECUTABLE	LOC_TOTAL, NUMBER_OF_LINES
BRANCH_COUNT, CYCLOMATIC_COMPLEXITY, MODIFIED_CONDITION_COUNT, DECISION_COUNT, CONDITION_COUNT, MULTIPLE_CONDITION_COUNT, EDGE_COUNT	CYCLOMATIC_COMPLEXITY	BRANCH_COUNT, MODIFIED_CONDITION_COUNT, DECISION_COUNT, CONDITION_COUNT, MULTIPLE_CONDITION_COUNT, EDGE_COUNT
GLOBAL_DATA_COMPLEXITY (correlated with CYCLOMATIC_COMPLEXITY)	*(already included above)*	GLOBAL_DATA_COMPLEXITY
HALSTEAD_CONTENT, NUM_UNIQUE_OPERANDS	HALSTEAD_CONTENT	NUM_UNIQUE_OPERANDS

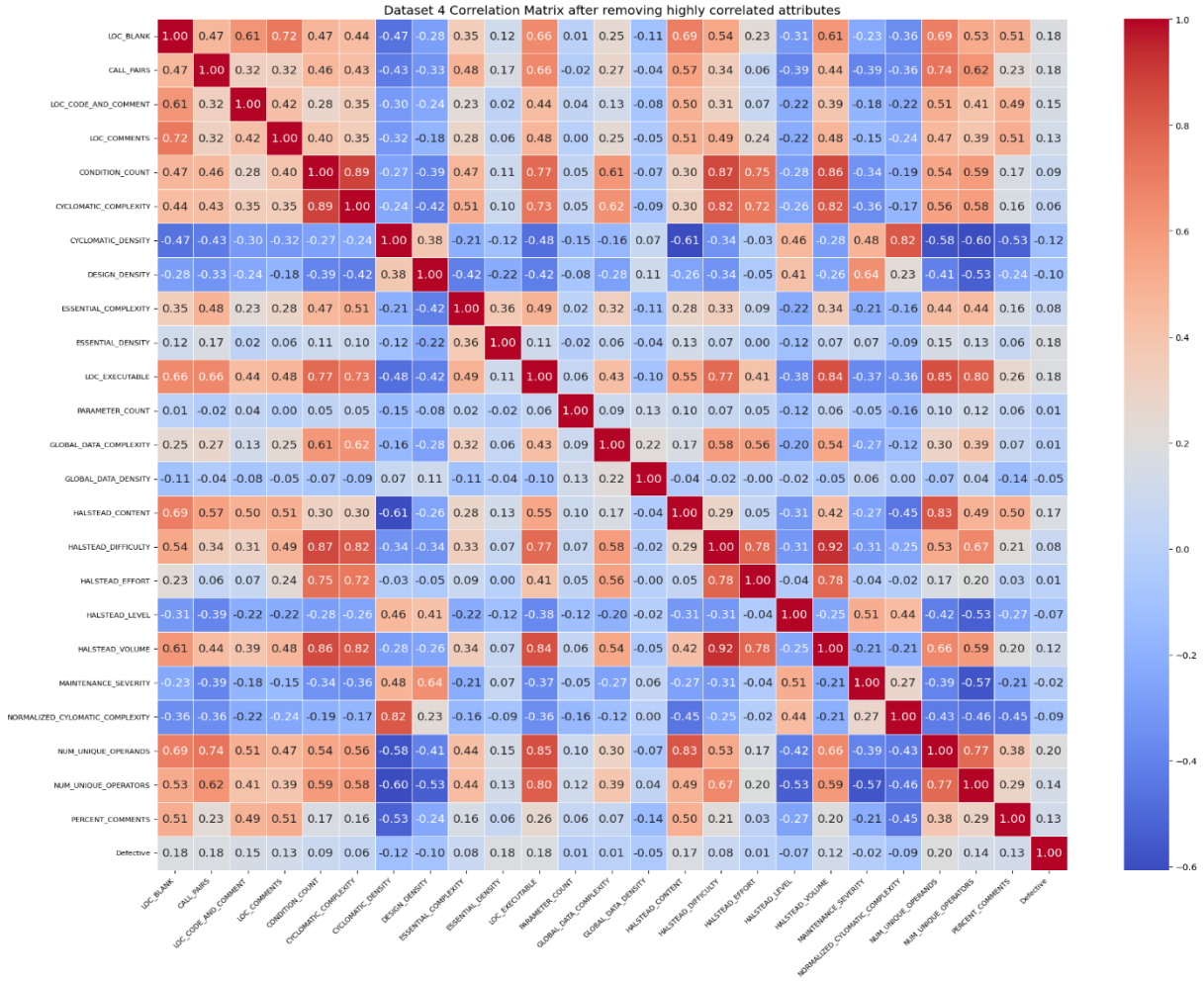
Table 3: *Feature Reduction Based on Correlation Analysis for Dataset 3*




Dataset 4 Correlation Matrix
LOC_BLANK
BRANCH_COUNT
CALL_PAIRS
LOC_CODE_AND_COMMENT
LOC_COMMENTS
CONDITION_COUNT
CYCLOMATIC_COMPLEXITY
CYCLOMATIC_DENSITY
DECISION_COUNT
DESIGN_COMPLEXITY
DESIGN_DENSITY
EDGE_COUNT
ESSENTIAL_COMPLEXITY
ESSENTIAL_DENSITY
LOC_EXECUTABLE
PARAMETER_COUNT
HALSTEAD_CONTENT
HALSTEAD_DIFFICULTY
HALSTEAD_EFFORT
HALSTEAD_ERROR_EST
HALSTEAD_LENGTH
HALSTEAD_LEVEL
HALSTEAD_PROG_TIME
HALSTEAD_VOLUME
MAINTENANCE_SEVERITY
MODIFIED_CONDITION_COUNT
MULTIPLE_CONDITION_COUNT
NODE_COUNT
NORMALIZED_CYLOMATIC_COMPLEXITY
NUM_OPERANDS
NUM_OPERATORS
NUM_UNIQUE_OPERANDS
NUM_UNIQUE_OPERATORS
NUMBER_OF_LINES
PERCENT_COMMENTS
LOC_TOTAL
Defective
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
−0.4
−0.6

GROUP	KEEP	DROP
HALSTEAD_EFFORT, HALSTEAD_PROG_TIME	HALSTEAD_EFFORT	HALSTEAD_PROG_TIME
CONDITION_COUNT, MODIFIED_CONDITION_COUNT, MULTIPLE_CONDITION_COUNT, DECISION_COUNT, EDGE_COUNT, NODE_COUNT, BRANCH_COUNT	CONDITION_COUNT	MODIFIED_CONDITION_COUNT, MULTIPLE_CONDITION_COUNT, DECISION_COUNT, EDGE_COUNT, NODE_COUNT, BRANCH_COUNT
HALSTEAD_ERROR_EST, HALSTEAD_VOLUME, HALSTEAD_LENGTH, NUM_OPERATORS, NUM_OPERANDS	HALSTEAD_VOLUME	HALSTEAD_ERROR_EST, HALSTEAD_LENGTH, NUM_OPERATORS, NUM_OPERANDS
LOC_EXECUTABLE, LOC_TOTAL, NUMBER_OF_LINES	LOC_EXECUTABLE	LOC_TOTAL, NUMBER_OF_LINES
CYCLOMATIC_COMPLEXITY, DESIGN_COMPLEXITY	CYCLOMATIC_COMPLEXITY	DESIGN_COMPLEXITY

Table 4: *Feature Reduction Based on Correlation Analysis for Dataset 4*



5.2. Performance Metrics

Performance metrics are critical for evaluating how well classification models perform. In this project, where the goal is to predict whether a software module is defective or not defective, these metrics help assess the quality of predictions beyond simple accuracy, especially when the classes are imbalanced.

5.2.1. Confusion Matrix Basics

Most classification metrics are derived from the confusion matrix, which breaks down predictions into four categories:

	Actual: Defective (1)	**Actual: Not Defective (0)**
Predicted: Defective (1)	TP (True Positive) – Correctly predicted defect	FP (False Positive) – Incorrectly flagged as defective
Predicted: Not Defective (0)	FN (False Negative) – Missed actual defect	TN (True Negative) – Correctly predicted as non-defective

Table 5: *Confusion Matrics*

Definitions:

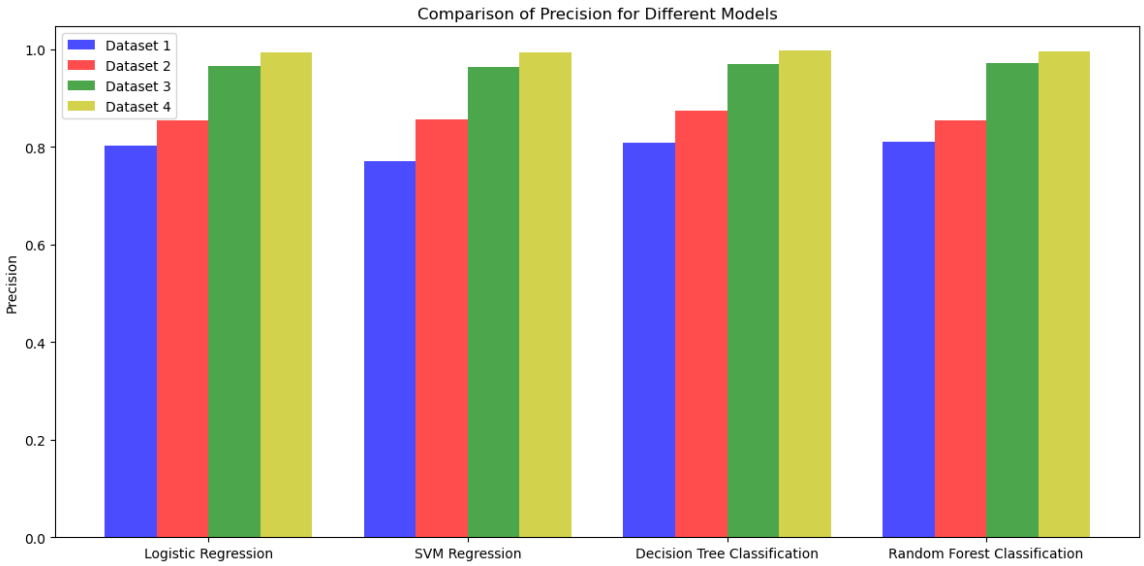
- True Positive (TP): Model correctly predicts a defective module.
- True Negative (TN): Model correctly identifies a non-defective module.
- False Positive (FP): Model incorrectly predicts a defect in a non-defective module.
- False Negative (FN): Model misses a defective module.

5.2.2. Precision

Precision is a metric that measures the accuracy of the positive predictions made by a classification model.

Formula:

$$Precision = \frac{TP}{TP + FP}$$

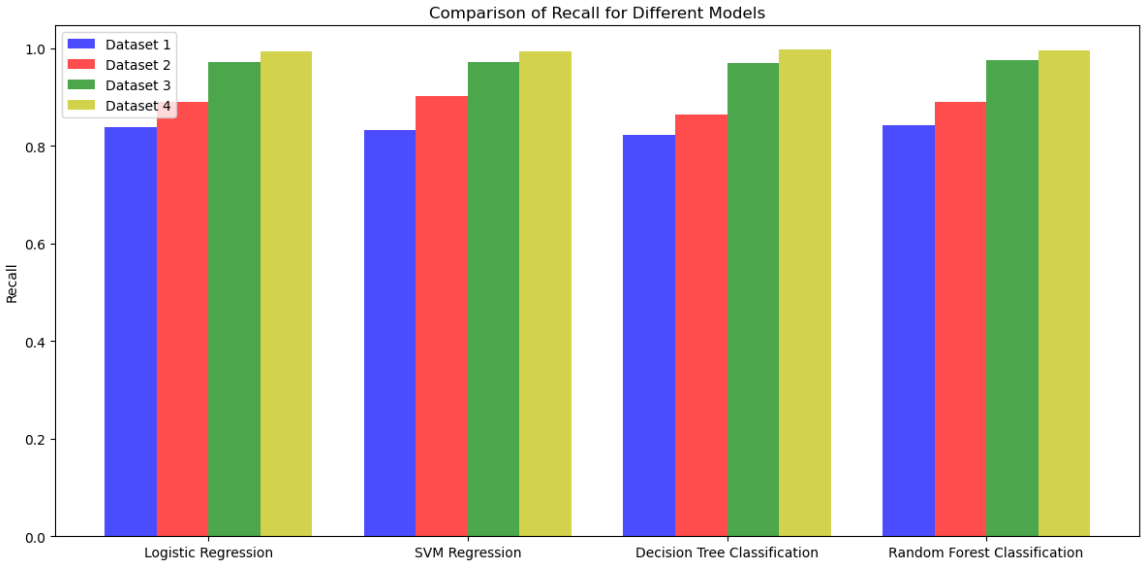


5.2.3. Recall

Recall measures the model's ability to find all actual positive cases.

Formula:

$$Recall = \frac{TP}{TP + FN}$$

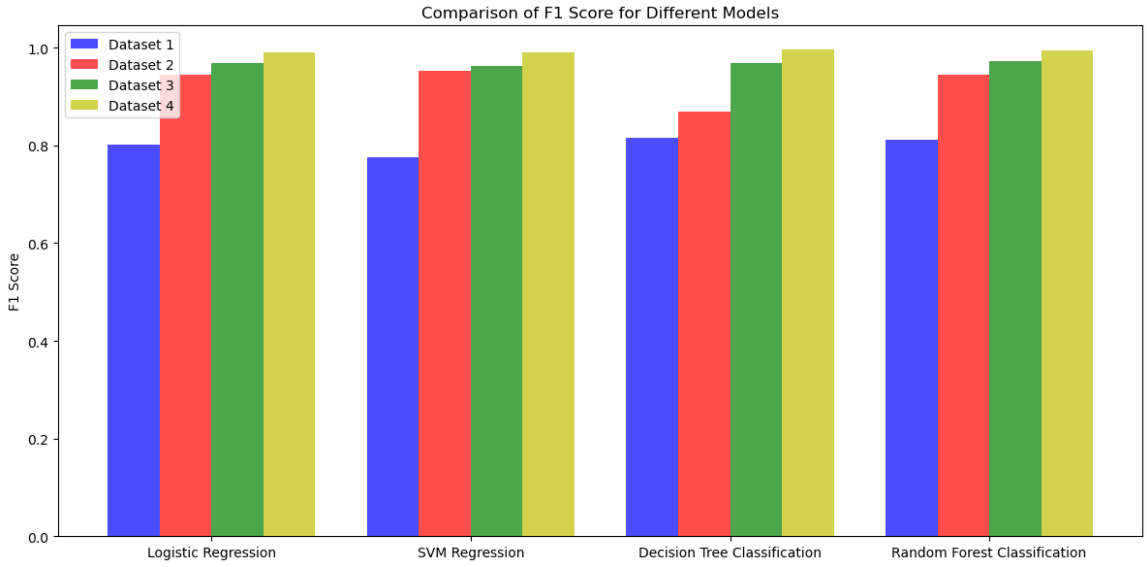


5.2.4. F1 Score

The F1 Score is the harmonic mean of Precision and Recall. It provides a single metric that balances both, useful when you want a fair evaluation of both false positives and false negatives.

Formula:

$$F1\ Score = 2 * \frac{Precision * Recall}{Precision + Recall}$$



5.2.5. MSE (Mean Squared Error)

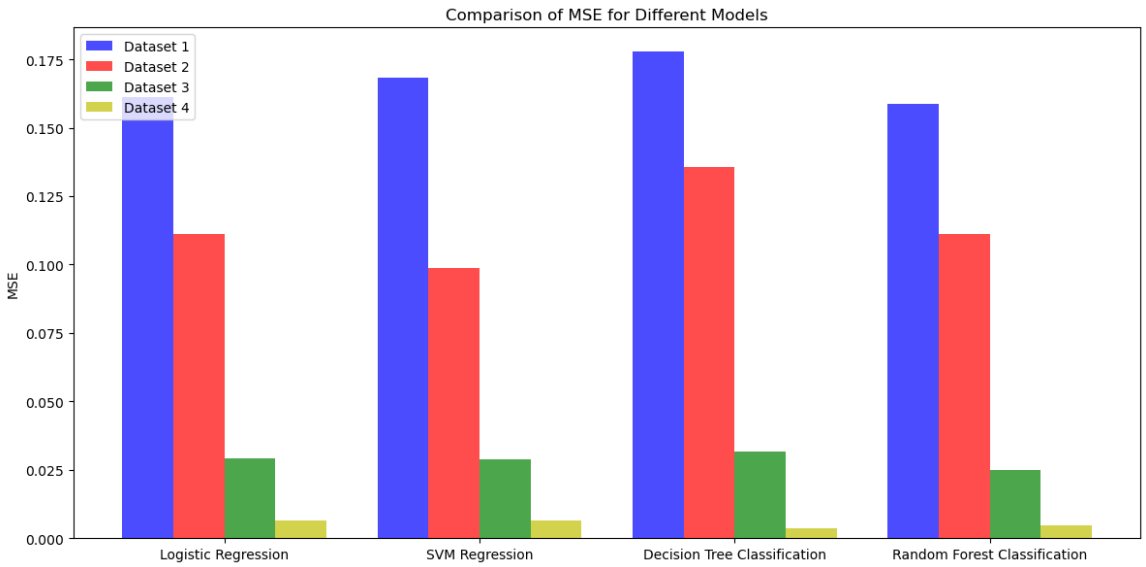
Mean Squared Error is a metric used to evaluate models. It measures the average of the squared differences between the predicted values and the actual values. A lower MSE indicates that the model's predictions are closer to the true values.

Formula:

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Where:

- n = Number of data points
- y_i = Actual value
- $\hat{y}_i$ = Predicted value



5.3. Comparison of values of each parameter in different models

5.3.1. DATASET 1 Dataset Results

Metric	Logistic Regression	SVM	Decision Tree Classification	Random Forest Classification
Precision	0.80115	0.769925	0.808925	0.80944
Recall	0.838863	0.831754	0.822275	0.841232
F1 Score	0.801022	0.775415	0.81464	0.812035
MSE	0.161137	0.168246	0.177725	0.158768

5.3.2. DATASET 2 Dataset Results

Metric	Logistic Regression	SVM	Decision Tree Classification	Random Forest Classification
Precision	0.854701	0.855602	0.873945	0.854701
Recall	0.888889	0.901235	0.864198	0.888889
F1 Score	0.945534	0.9519	0.868965	0.945534
MSE	0.111111	0.0987654	0.135802	0.111111

5.3.3. **DATASET 3 Dataset Results**

Metric	Logistic Regression	SVM	Decision Tree Classification	Random Forest Classification
Precision	0.965785	0.963216	0.969611	0.971878
Recall	0.970913	0.971204	0.968296	0.974985
F1 Score	0.967578	0.961595	0.968926	0.972957
MSE	0.0290867	0.0287958	0.0317045	0.0250145

5.3.4. **DATASET 4 Dataset Results**

Metric	Logistic Regression	SVM	Decision Tree Classification	Random Forest Classification
Precision	0.993704	0.993704	0.996318	0.994525
Recall	0.993664	0.993664	0.996304	0.995248
F1 Score	0.990506	0.990506	0.995547	0.994274
MSE	0.0063358	0.0063358	0.00369588	0.00475185

5.4. ROC Curve and AUC

5.4.1. ROC (Receiver Operating Characteristic) Curve

The ROC Curve is a graphical plot used to evaluate the performance of a binary classification model. It shows the trade-off between the True Positive Rate (TPR) and False Positive Rate (FPR) at various classification thresholds.

- True Positive Rate (TPR): Also called Recall or Sensitivity.

$$TPR = \frac{TP}{TP + FN}$$

- False Positive Rate (FPR):

$$FPR = \frac{FP}{FP + TN}$$

Interpretation:

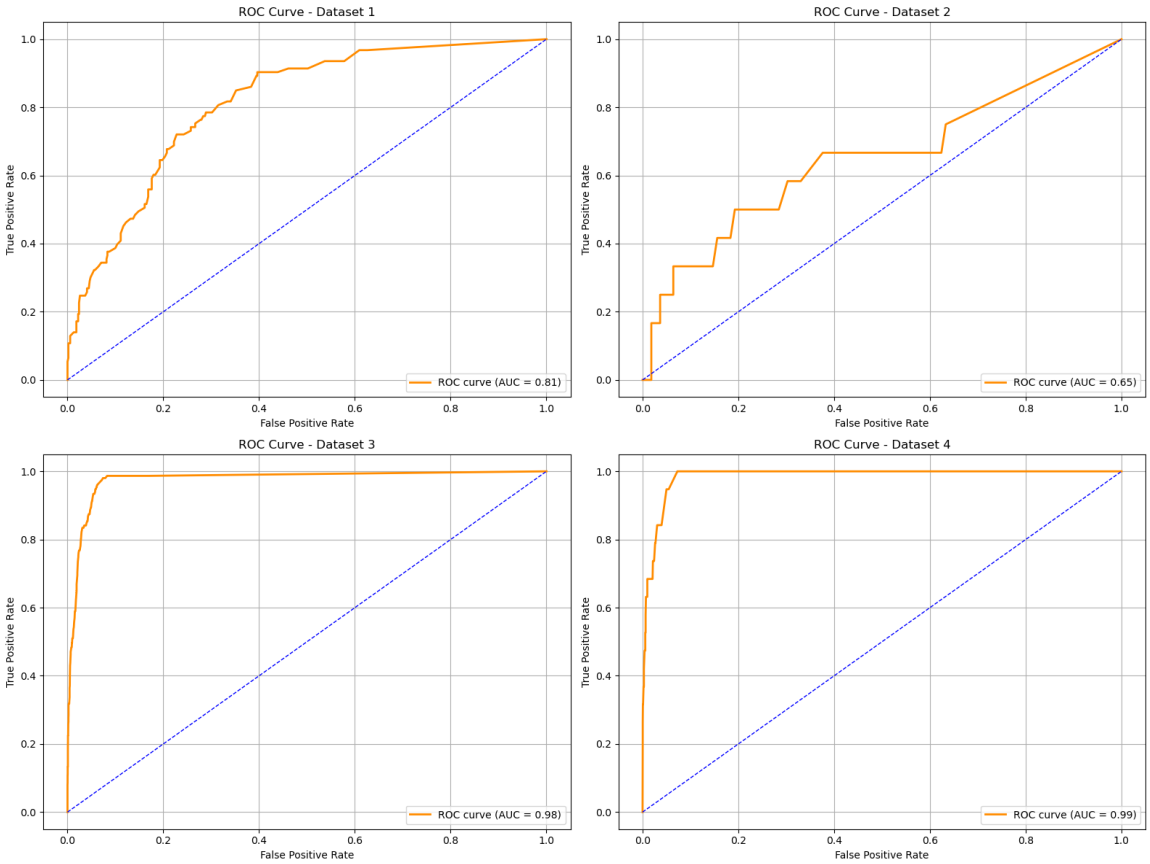
- The closer the curve follows the top-left corner, the better the model is at distinguishing between the classes.
- A diagonal line represents random guessing (AUC = 0.5).
- A perfect classifier has an ROC curve that reaches the top-left corner (AUC = 1.0).

5.4.2. AUC – Area Under the Curve

AUC stands for Area Under the Curve, specifically under the ROC Curve. It is a single scalar value that measures the overall performance of a binary classification model across all classification thresholds.

- AUC = 1: Perfect prediction
- AUC = 0.5: No discrimination (random)
- AUC < 0.5: Worse than random

5.4.3. Interpretation of ROC Curve for various datasets



5.4.4. Value of AUC for all four datasets

- AUC for dataset 1: 0.8127
- AUC for dataset 2: 0.6498
- AUC for dataset 3: 0.9764
- AUC for dataset 4: 0.9860

CHAPTER 6

CONCLUSION

This research explored the application of supervised machine learning techniques for predicting software faults using static code metrics extracted from real-world software datasets. As modern software systems become increasingly complex, early identification of potential defects is crucial to ensuring software quality, minimizing risk, and reducing maintenance costs.

Four widely used classification algorithms like Logistic Regression, Support Vector Machine (SVM), Decision Tree, and Random Forest, were evaluated across four benchmark datasets: DATASET 1, DATASET 2, DATASET 3, and DATASET 4. A comprehensive data preprocessing pipeline was developed, incorporating outlier handling using the interquartile range (IQR), feature correlation analysis for dimensionality reduction, and imputation techniques for handling missing values. The classifiers were assessed using standard evaluation metrics: Precision, Recall, F1 Score, and Mean Squared Error (MSE).

Among the models, the Random Forest classifier consistently demonstrated superior performance across most datasets. In the DATASET 1 dataset, it achieved a precision of 0.809, recall of 0.841, and MSE of 0.158, outperforming the other models. On the DATASET 2 dataset, it delivered a strong F1 Score of 0.9455 and MSE of 0.111, performing on par with SVM and Logistic Regression. The DATASET 3 dataset showed the best results for Random Forest, with the highest F1 Score (0.9729) and lowest MSE (0.025), confirming its effectiveness on large-scale data. For the DATASET 4 dataset, it achieved near-perfect precision and recall (both above 0.99), closely matching the performance of the Decision Tree classifier.

This research highlights the importance of data preprocessing and feature selection in improving the performance of machine learning models for software fault prediction. The findings suggest that, with appropriate handling of data and the right choice of algorithms, machine learning can significantly contribute to early-stage defect detection, ultimately supporting higher software reliability and efficient development practices.

REFERENCE

1. Khoshgoftaar, T. M., & Szabo, R. M. (1996). Using neural networks to predict software faults during testing. *IEEE Transactions on Reliability*, *45*(3), 456-462.
2. Radjenović, D., Heričko, M., Torkar, R., & Živkovič, A. (2013). Software fault prediction metrics: A systematic literature review. *Information and software technology*, *55*(8), 1397-1418.
3. Malhotra, R. (2015). A systematic review of machine learning techniques for software fault prediction. *Applied Soft Computing*, *27*, 504-518.
4. Wahono, R. S. (2015). A systematic literature review of software defect prediction. *Journal of software engineering*, *1*(1), 1-16.
5. Bolón-Canedo, V., Sánchez-Maroño, N., & Alonso-Betanzos, A. (2015). Recent advances and emerging challenges of feature selection in the context of big data. *Knowledge-based systems*, *86*, 33-45.
6. Nagappan, N., & Ball, T. (2005, May). Use of relative code churn measures to predict system defect density. In *Proceedings of the 27th international conference on Software engineering* (pp. 284-292).
7. Lessmann, S., Baesens, B., Mues, C., & Pietsch, S. (2008). Benchmarking classification models for software defect prediction: A proposed framework and novel findings. *IEEE transactions on software engineering*, *34*(4), 485-496.
8. Aquil, Ishak, Predicting software defects using machine learning techniques, IJATCSE 9 (4) (2020) 6609‑6616.
9. Tang, Y., Dai, Q., Yang, M., Du, T., & Chen, L. (2023). Software defect prediction ensemble learning algorithm based on adaptive variable search algorithm. *International Journal of Machine Learning and Cybernetics*, *14*(6), 1967-1987.
10. Ghotra, B., McIntosh, S., & Hassan, A. E. (2015, May). Revisiting the impact of classification techniques on the performance of defect prediction models. In *2015*

IEEE/ACM 37th IEEE International Conference on Software Engineering (Vol. 1, pp. 789-800). IEEE.

11. Yao, J., & Shepperd, M. (2020, April). Assessing software defection prediction performance: Why using the Matthews correlation coefficient matters. In *Proceedings of the 24th International Conference on Evaluation and Assessment in Software Engineering* (pp. 120-129).

12. M. Belouch, S. Elhadaj, M. Idhammad, A hybrid filter-wrapper feature selection method for DDoS detection in cloud computing, Intell. Data Anal. 22 (2018) 1209‑1226.

13. Matloob, F., Ghazal, T. M., Taleb, N., Aftab, S., Ahmad, M., Khan, M. A., ... & Soomro, T. R. (2021). Software defect prediction using ensemble learning: A systematic literature review. *IEEe Access*, *9*, 98754-98771.

14. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *the Journal of machine Learning research*, *12*, 2825-2830.

15. Malhotra, R. (2015). A systematic review of machine learning techniques for software fault prediction. *Applied Soft Computing*, *27*, 504-518.

16. McKinney, W. (2010). Data structures for statistical computing in Python. *SciPy*, *445*(1), 51-56.

17. K.S. Kavya, Y. Prasanth, An ensemble DeepBoost classifier for software defect prediction, Int. J. Adv. Trends Comput. Sci. Eng. 9 (2) (2020) 2021‑2028.

18. M. Kondo, C.P. Bezemer, Y. Kamei, A.E. Hassan, O. Mizuno, The impact of feature reduction techniques on defect prediction models, Empir. Softw. Eng. (2019) 1‑39.

19. S. Nakariyakul, High-dimensional hybrid feature selection using interaction information-guided search, Knowl. Based Syst. 145 (2018) 59‑66.

20. NASA Metrics Data Program (MDP) datasets. (2010+). https://mdp.ivv.nasa.gov/

21. Geeks for geeks https://www.geeksforgeeks.org/